

Machine Learning Week 9 Assignment

By Gaurav Naudiyal (Student ID: 25339213)

Msc Computer Science – Augmented & Virtual Reality

(i) (a) for child speech datasets (training and test)

Content : Both input_childSpeech_trainingSet.txt and input_childSpeech_testSet.txt consists of short and simple phrases. Most are a few words long. It also contain a lot of repetitions and topics are said like snacks (“I want cookie”), bedtime(“Night night”) or simple requests like (“Help me please”, “I wash hands”, “I want play”)

Vocabulary : The vocabulary that is used is very basic and the words are like words which children use like “mama”, “juice”, “teddy”, “night”, “cookie”, “doggy”. These words appear often in the datasets. The phrases are mostly gentle and positive and said in first person.

Length : For training set, the dataset has approximately 10,000 lines and 55,000 words. On the other hand for the test set, the dataset has about 1,000 lines and approx 5,000 words making it 1/10th of the training set.

For shakespeare dataset

Content : this dataset is much longer than the child speech datasets. The dataset appears to contain dialogue structure from a play or a dramatic script with characters speaking in Old or Early Modern English. The text in this one is much more complex and poetic.

Vocabulary : The vocabulary size is much larger here, consisting of the diverse, expressive language used in Shakespeare’s works.

Length : It has about 40,000 lines and contains approximately 200,000 words in total.

(b) 1. N_embd : It is currently set to 384, n_embd affects the dimensionality of each token's vector representation. Reducing this will have the largest impact as it reduces the embedding table size, and shrinks the dimensionality of all fully connected layers, projection matrices and the model's overall memory footprint. Lowering it to 160 would make it still capture sufficient feature richness but with lesser parameters.

2. n_layer : This means the number of layers and it is currently set to 6 and it dictates the depth of the transformer model. Reducing it from 6 to 3 will lower the depths of the model, meaning fewer blocks and fewer parameters per block, making the model shallower and faster without sacrificing depth fully.

3. n_head: This is the number of attention heads and is currently set to 6 like n_layer, in this each head requires its own set of attention weights. If reduced from 6 to 4, it would lower the number of attention heads resulting in fewer attention matrices and projections, while keeping the model's attention mechanisms expressive enough.

4. block_size : This is currently set to 256 and determines the maximum context length for each sequence. Reducing the block size to 96 might not affect the parameter count directly but it can significantly speed up training and make experiments more manageable for short sequence data.

5. batch_size : This parameter doesn't directly affect model parameters, reducing it will speed up training and lower memory consumption.

6. dropout : This parameter doesn't directly affect the model parameters, but reducing it can slightly improve performance.

Model parameters : 10.769704 M parameters (max_iters = 1000)

(c)

I have set eval_interval = 100 to get more steps

Configuration 1 :

n_embd : 144

n_layer : 4

n_head : 4

block_size : 128

1.031368 M parameters

step 0: train loss 3.7122, val loss 3.7140
step 100: train loss 1.9669, val loss 1.9767
step 200: train loss 0.7443, val loss 0.7493
step 300: train loss 0.4083, val loss 0.4115
step 400: train loss 0.3762, val loss 0.3786
step 500: train loss 0.3640, val loss 0.3669
step 600: train loss 0.3587, val loss 0.3609
step 700: train loss 0.3542, val loss 0.3577
step 800: train loss 0.3540, val loss 0.3564
step 900: train loss 0.3539, val loss 0.3569
step 999: train loss 0.3505, val loss 0.3528

Configuration 2:

n_embd: 160
n_layer: 3
n_head: 4
block_size: 96

0.95492 M parameters

step 0: train loss 3.7532, val loss 3.7512
step 100: train loss 1.8469, val loss 1.8579
step 200: train loss 0.5491, val loss 0.5531
step 300: train loss 0.3962, val loss 0.3999
step 400: train loss 0.3803, val loss 0.3833
step 500: train loss 0.3733, val loss 0.3741
step 600: train loss 0.3670, val loss 0.3699
step 700: train loss 0.3620, val loss 0.3666
step 800: train loss 0.3622, val loss 0.3651
step 900: train loss 0.3621, val loss 0.3641
step 999: train loss 0.3585, val loss 0.3613

Configuration 3 :

n_embd: 192
n_layer: 2
n_head: 4
block_size: 64

0.916648 M parameters

step 0: train loss 3.7265, val loss 3.7274
step 100: train loss 1.5933, val loss 1.6069
step 200: train loss 0.4594, val loss 0.4642
step 300: train loss 0.4061, val loss 0.4097
step 400: train loss 0.3951, val loss 0.3984
step 500: train loss 0.3881, val loss 0.3899
step 600: train loss 0.3857, val loss 0.3888
step 700: train loss 0.3805, val loss 0.3828
step 800: train loss 0.3793, val loss 0.3843
step 900: train loss 0.3763, val loss 0.3803
step 999: train loss 0.3778, val loss 0.3807

Validation loss : Validation loss is a key indicator of model's generalization capabilities. The following are final validation losses:

Configuration 1: 0.3528
Configuration 2: 0.3613
Configuration 3: 0.3807

Configuration 1 has the lowest validation loss which means the best generalization to unseen data. Configuration 2 comes next and it represents a balance between model complexity and size. Configuration 3 has the highest validation loss indicating the worst generalization out of the 3.

Overfitting : Overfitting can be achieved by comparing the gap between training loss and validation loss.

Configuration 1 : Train loss is 0.3505 and validation loss is 0.3525, indicating gap of 0.0020

Configuration 2 : Train loss is 0.3585 and val loss is 0.3613 creating gap of 0.0028
Configuration 3 : Train loss is 0.3778 and val loss is 0.3807 creating gap of 0.0029

All the configurations show minimal overfitting but configuration one shows the smallest gap showing the best balance between model size and regularization. Configuration 2 and 3 are very close in terms of gap with only 0.0001 separating them.

Qualitative assessment of Generated Output:

Configuration 1 :

I cliRbook bed Read book
Come here Where ball
I run fast Daddy play
Read book k Big hug
More more Read book
Come here I want cooki I jump
All done I hungry
I run fast I did it
Read book More bubbles
No luch No st ke it
All done
Night night Uh oh spill
Night night More bubbles
I tired Help me pplease
I want more Daddy read me dinosaur book before bed
All done I hungry
Night night night
Big hug Big truck I drajuck
No like it Where kitty go
I see doggy My More more
Saw big flufffy ddoggy at park it

The sentences in configuration 1 have reasonable fluency. The output includes frequent phrases like “night night” “doggy” “read book” “more more” etc. The phrases are in a childish manner. But sometimes there are some minor

deviations like “I cliRbook” instead of “I read book”, “I want cooki” instead of “I want cookie”.

Configuration 2 :

What is th Come here
Bath time Shoes on
I wash hands I hide
Uh oh spill No stop
All done Daddy read me dinosaur book beforet
I run fast I hungry
Big hug All gone
Daddy read me dinosaur book before bed No mine
I make mess that
Help me please Shoes ohes it
I want more juice please Bath time
I want that I want cookie
I found it
I want more juice please Bath time
I draw Read book
Read book Big hug
Look moon I sive you
I drawCome here
What is that Bath time
I want cookie I want cookie
Come here I jum

Similarly to configuration 1, the sentences have resonable fluency. The words are short and concise. Phrases like “Daddy read me dinosaur book before bed” indicate good retention of thematic elements from the training set but some phrases like “shoes ohes” show that the model capacity to generalize has reduced.

Configuration 3 :

I climb I jump
No nap Look moon
Help me please I found it

hugh Daddy read me dinosaur book before bed
I run fast I want more juice please
All gone I make mess
I did it Saw big fluffy doggy at park it run fast
Help me please I love you
No pilike it Where ball
Look airplane What is that
Daddy read me dinosaur book bell
Where ball Bath time
I jump Where kitty go
I w
No mine What I hide
I run fast Saw big fluffy doggy at park it run fast
I see doggy Where ball
My teddy I want that
I tired I want coo

Out of all the 3 configurations, configuration 3 shows the worst flow of sentences. There are some lines that are incomplete like “I w” “hugh daddy” “I want coo” “No pilike”. It lacks richness in vocabulary but there is still basic coherence.

(d) Impact of including bias terms:

Advantages :

- It added a little freedom to self-attention layer and it makes it adapt better to different patterns in the data, especially when the inputs are a bit weird. This is very handy in cases where the model sees a lot of variety.
- It also gave the network a better gradient flow resulting in gradients not being stuck at zero.

Disadvantages :

- It added more parameters. Everytime a bias is added, the model learns new values. This slightly increases the overall size of the model.
- With extra flexibility it also increases the risk of overfitting.

Impact on performance :

With bias:

- Adding bias makes the model adjust its attention even if the inputs aren't super clear or all pretty similar so it can highlight important tokens.
- It improves generalization
- It also improves noise handling

Without bias:

- Skipping the bias will make the model only use what's there in the input and it might struggle if the data provided is weak
- With less flexibility, there is a chance that the model might generalize a bit better

(e)

Impact of skip connections:

Without skip connections: Training was way less stable, there was slower convergence and instability. The outputs were less coherent and it resulted in a higher risk of overfitting.

With skip connections: Training was much more reliable. The convergence was much faster and it was more stable. The sentences/phrases were much more coherent and it resulted in better regularization which also improves generalization.

- (ii) (a) Best model was configuration 1 from (i) and it reported the test losses as follows :

1.033096 M parameters

step 0: train loss 3.6844, val loss 3.6840

step 100: train loss 1.9794, val loss 1.9867
step 200: train loss 0.8542, val loss 0.8608
step 300: train loss 0.4053, val loss 0.4066
step 400: train loss 0.3752, val loss 0.3763
step 500: train loss 0.3661, val loss 0.3684
step 600: train loss 0.3628, val loss 0.3660
step 700: train loss 0.3571, val loss 0.3607
step 800: train loss 0.3564, val loss 0.3603
step 900: train loss 0.3503, val loss 0.3536
step 999: train loss 0.3512, val loss 0.3531
Saved best model at step 999 with val loss 0.3531
Test loss on input_childSpeech_testSet.txt: 0.3600

The test loss was 0.3600. This is close to the final validation loss which means that the model generalized well and did not overfit. If we compare this to a dummy or random baseline, it is a very strong improvement. However, the dataset's simplicity and repetition might make it easier for the model to memorize patterns rather than truly generalize so it can make the user overestimate its capabilities.

(b) 1.040321 M parameters

step 0: train loss 4.2604, val loss 4.2624
step 100: train loss 2.0783, val loss 2.0836
step 200: train loss 0.8112, val loss 0.8142
step 300: train loss 0.4094, val loss 0.4115
step 400: train loss 0.3749, val loss 0.3783
step 500: train loss 0.3683, val loss 0.3710
step 600: train loss 0.3615, val loss 0.3633

step 700: train loss 0.3575, val loss 0.3611

Saved best model at step 800 with val loss 0.3599

step 900: train loss 0.3535, val loss 0.3557

step 999: train loss 0.3498, val loss 0.3537

Saved best model at step 999 with val loss 0.3537

Test loss on input_shakespeare.txt: 6.6025

The loss is extremely high compared to the test loss in the child speech. A loss above 6 generally means that the model is guessing randomly and cannot predict the Shakespeare language.

A random or dummy model would also score similarly. Since the model was only trained with the child data, it has zero knowledge about the Shakespeare vocabulary, writing and structure so it is unable to generalize.

The huge gap between the two test losses shows that our model has learnt simple and repetitive style of child speech well but it cannot transfer its knowledge to Shakespeare writing which makes sense because it doesn't have exposure to such writing style.

In practice, this pipeline shows that such models are very domain specific meaning that they work well generating or evaluating language that they saw during the training. If we wanted a model that could also handle the Shakespeare language, we would have to include samples from both styles in the training data or train separate models for each writing style.